

Foundations for programming languages 4: Parameter passing, Scope rules

Isao Sasano

How do we get to values from variables?

In imperative languages variables represent locations (via their declarations). By accessing the locations we get the values. How we relate an occurrence of a variable, its declaration, and its location is specified by the scope rule and the parameter passing method of the language.

- Parameter passing methods
 - call by value, call by reference, call by name
- Correspondence between an occurrence of a variable and its declaration
 - Static scope, dynamic scope

Procedures

Procedure is a construct for giving a name to a piece of code. The piece is referred to as the *procedure body*. When the name is called, the body is executed. Each execution of the body is called an *activation*.

Function is a procedure that returns a value. People may not distinguish between procedures and functions.

Procedure calls and function calls

A function call is an expression, while a (non-function) procedure call is a statement.

(ex.) `r * sin (angle)`

`sin (angle)` is an function call expression. It can appear anywhere an expression can appear as far as the syntax is concerned.

(ex.) `read (ch)`

`read (ch)` is a procedure call statement. It can appear anywhere a statement can appear as far as the syntax is concerned.

Syntax of procedure calls

Syntax of procedure (or function) calls (in prefix notation)

`<procedure name> ( <parameters> )`

The parameters are called *actual parameters*.

(ex.) angle is an actual parameter in the function call
sin (angle).

(ex.) ch is an actual parameter in the procedure call
read (ch).

In procedure calls, parentheses are usually necessary even if there is no parameters (in C, Modula-2, Java, etc.). In Pascal we do not write parentheses if there is no parameters.

(ex.) **begin while** eoln **do** readln; read(ch) **end**

Eoln is a function call expression, readln is a procedure call statement, and read(ch) is a procedure call statement.

Syntax of procedure declarations

Procedure (or function) declarations consists of the following.

- Name of the procedure
- Names and types of parameters
 - The parameters are called *formal parameters*.
- Result type (in function declarations)
- Local declarations and statements

Designers of languages freely define the syntax of procedure declarations as far as the above four are clear.

An procedure declaration in Pascal

```
procedure getch;  
  begin  
    read (ch)  
  end;
```

The name of the above procedure is getch.
It has no parameters.

A function declaration in Pascal

```
function f (x : integer) : integer;  
var square : integer;  
begin  
    square := x * x;  
    f := square + 1  
end;
```

The name of the function is f. The parameter has the type of integer and the result type is integer. In Pascal, the assignment `f:= ...` determines the return value of the function.

Usual languages like C, Modula-2, Java, etc. provide return statements.

Recursive functions

```
function f (n : integer) : integer;  
begin  
    if n = 0 then f := 1 else f := n * f (n-1);  
end;
```

The function f computes the factorial of the given parameter. For example, the computation of f(3) can be illustrated as follows.

$$\begin{aligned} f(3) &= 3 * f(2) && = 6 \\ f(2) &= 2 * f(1) && = 2 \\ f(1) &= 1 * f(0) && = 1 \\ f(0) &= 1 \end{aligned}$$

Parameter passing methods

```
function square (x : integer) : integer;  
begin  
    square := x * x  
end;
```

In the function x is a parameter.

For example, the value of a function call expression `square(2)` is 4, which is obtained by evaluating `x * x` in the state where 2 is assigned to x. The situation is simple when the actual parameter is a number, but there are various ways when the actual parameter is a variable or an element of an array.

Parameter passing methods

There are three major ways for parameter passing.

- Call-by-value
- Call-by-reference
- Call-by-name

Call by value

The value of actual parameters are passed to the corresponding formal parameters. Suppose a procedure (or function) p has a formal parameter x . The execution (or evaluation) of procedure call $p(e)$ is performed as follows.

- (1) $x := e$

- (2) execute the body of the procedure p

- (3) return the result (when p is a function).

(Note) When the variable x is also declared in the caller, the formal parameter x is different from that.

(ex.) When evaluating `square (2+3)`, $x := 2 + 3$ is firstly performed, thus 5 is assigned to the formal parameter x . Then $x * x$ is evaluated to 25, which is the value of the function call expression `square (2+3)`.

An example that does not work

```
procedure nget (c : char);  
  begin  
    read (c)  
  end;
```

When this procedure is called, a keyboard input is assigned to the parameter c. When executing `nget (ch)`, the value of `ch` is not affected.

Another example that does not work

```
procedure swap (x : integer; y : integer);  
var z : integer;  
  begin  
    z := x; x := y; y := z  
  end;
```

Executing the procedure call swap (a,b) does not change the value of variables a and b.

The values of a and b are assigned to the formal parameters x and y respectively and the the value of x and y are swapped, so the values of a and b are not affected.

Call by reference

A formal parameter becomes a synonym for the actual parameter. (The location of the formal parameter becomes the location of the actual parameter.)

Pascal has call by value and call by reference.

```
procedure p (x : integer; var y : integer);
```

...

Formal parameters with **var** is call by reference and ones without **var** is call by value.

In the above example, the second argument of p must be an expression that has some location (i.e., that can appear in the LHS of an assignment), such as a variable or an element of array.

Procedure for swapping in Pascal

```
procedure swap (var x : integer; var y : integer);  
var z : integer;  
begin  
    z := x; x := y; y := z  
end;
```

In swap, x and y are call by reference. For example, execution of swap (i, A[i]) is performed as follows.

- (1) Make the location of x same as that of i.
- (2) Make the location of y same as that of A[i].
- (3) z := x; x := y; y := z

Suppose the value of i is 2, the value of A[2] is 99. Then the execution of the procedure call is effectively same as the execution of z := 2; i := 99; A[2] = z, so the values of i and A[2] are swapped.

Exercise

Show the result (display output) when executing the following Pascal program. The procedure `writeln` prints the value of the parameter and a newline character.

```
program test;  
var x : integer;  
var y : integer;  
procedure swap  
  (var x: integer; var y : integer);  
var z : integer;  
begin  
  z := x; x := y; y := z  
end;  
  
begin  
  x := 3;  
  y := 4;  
  swap (x,y);  
  writeln (x);  
  writeln (y)  
end.
```

About the language C

The language C supports only the call by value as parameter passing. Instead, C provides pointers so that we can simulate call by reference by passing pointers to functions as their parameters.

```
void swap (int * px, int * py) {  
    int z;  
    z = *px; *px = *py; *py = z;  
}
```

The following program fragment swaps the values of variables a and b.

```
int a = 1, b = 2;  
swap (&a, &b);
```

Call by name

Actual parameters are textually substituted for the formal parameters. Name conflicts are avoided by renaming the local variables in the procedure body. Algol60 is call by name by default.

```
program {computation of inner product}
  var i, n, z : integer;
      a, b : array [0..9] of integer;
  procedure sum (x : integer);
      begin while i < n do begin z := z + x; i := i + 1 end end;
  begin n := 10; i := 0; z := 0; sum (a[i] * b[i]); writeln (z) end.
```

Arguments are evaluated when necessary. It is called *lazy evaluation*.
Lazy evaluation is call by name or *call by need* (an argument is evaluated once). If there is no side-effect, these two strategies yields the same result.

(supplement) Call-by-value-result

In Call-by-value-result (also called as copy-in/copy-out), the actual parameters are initially copied into the formals and the locations of the actuals are computed and saved. The final values of the formals are copied back out to the saved locations.

(ex.) **program** {a subtle example}

i, *j* : **integer**;

procedure foo (*x*, *y* : **integer**); **begin** *i* := *y* **end**;

begin *i* := 2; *j* := 3; foo (*i*, *j*) **end**

In call-by-reference the value of *i* would become 3, while in call-by-value-result the value of *i* (and *j*) would not be changed.

Ada has three ways, **in**, **out**, and **in out**, for parameter passing. **in out** is for call-by-value-result.

Scope rules

The scope rules of a language determine the correspondence between an occurrence of a name (such as variables, types, procedures, etc.) and its declaration.

In *static scope* (also called as *lexical scope*), scopes of names can be determined statically (i.e., in compile-time).

In *dynamic scope*, the binding of name occurrences to declarations is done dynamically (i.e., at run time).

An example

program L;	{body of the program L}
var n : char;	begin
procedure W;	n := 'L';
begin	W;
writeln(n)	D
end;	end.
procedure D;	
var n : char;	
begin	
n := 'D';	
W	
end;	

Static scope

In static scope, intuitively, an occurrence of a name x corresponds to the inner-most declaration of x . Almost all languages like Pascal and C are static scope. In static scope, the previous example would result in

L

L

Principle: Consistent renaming of local names in the source program should have no effect on the computation.

By following the principle, the scope rule becomes static scope.

Dynamic scope

Under dynamic scope, an occurrence of a name is bound to a declaration at run time. In dynamic scope, the previous example would result in

L

D

Emacs lisp uses dynamic scope.

(Idea) In dynamic scope, an occurrence of a name x corresponds to the x that is located in the closest activation record in the stack.